

pattern matches the whole string. In this case, the string 'Open Source' is just a substring of the string 'Magazine Open Source For You' and since there is no match, the method `matches()` returns `False`, and the `if` statement displays the message 'No Match Found' on the terminal.

If you replace the line of code:

```
'Pattern pat = Pattern.compile("Open Source");'
```

...with the line of code:

```
'Pattern pat = Pattern.compile("Magazine Open Source For You");'
```

...then you will get a match and the `matches()` method will return `True`. The file with this modification `Regex2.java` is also available for download. The line of code:

```
'System.out.println("Match from " + (mat.start() + 1) + " to " + (mat.end() ));'
```

...uses two methods provided by the `Matcher` class, `start()` and `end()`. The method `start()` returns the start index of the previous match and the method `end()` returns the offset after the last character matched. So, the output of the program `Regex2.java` will be 'Match from 1 to 28'.

Figure 3 shows the output of `Regex1.java` and `Regex2.java`. An important point to remember is that the indexing starts at 0 and that is the reason why 1 is added to the value returned by the method `start()` as `(mat.start() + 1)`. Since the method `end()` returns the index immediately after the last matched character, nothing needs to be done there.

The `matches()` method of `Pattern` class with this sort of a comparison is almost useless. But many other useful methods are provided by the class `Matcher` to carry out different types of comparisons. The method `find()` provided by the class `Matcher` is useful if you want to find a substring match.

```
# javac Regex1.java
# java Regex1
No Match Found
# javac Regex2.java
# java Regex2
Match from 1 to 28
```

Figure 3: Output of `Regex1.java` and `Regex2.java`

Replace the line of code:

```
'if(mat.matches() )'
```

...in `Regex1.java` with the line of code:

```
'if(mat.find() )'
```

...to obtain the program `Regex3.java`. On execution,

`Regex3.java` will display the message 'Match from 10 to 20' on the terminal. This is due to the fact that the substring 'Open Source' appears from the 10th character to the 20th character in the string 'Magazine Open Source For You'. The method `find()` also returns `True` in case of a match and `False` in case if there is no match. The method `find()` can be used repeatedly to find all the matching substrings present in a string. Consider the program `Regex4.java` shown below.

```
import java.util.regex.*;
```

```
class Regex4
```

```
{
    public static void main(String args[])
    {
        Pattern pat = Pattern.compile("abc");
        String str = "abcdabcdabcd";
        Matcher mat = pat.matcher(str);
        while(mat.find() )
        {
            System.out.println("Match from " + (mat.start() + 1) + " to " + (mat.end() ));
        }
    }
}
```

In this case, the method `find()` will search the whole string and find matches at positions starting at the first, fifth and ninth characters. The line of code:

```
'String str = "abcdabcdabcd";'
```

...is used to store the string to be searched, and in the line of code:

```
'Matcher mat = pat.matcher(str);'
```

...this string is used by the method `matcher()` for further processing. Figure 4 shows the output of the programs `Regex3.java` and `Regex4.java`.

Now, what if you want the matched string displayed instead of the index at which a match is obtained. Well, then you have to use the method `group()` provided by the class

```
# javac Regex3.java
# java Regex3
Match from 10 to 20
# javac Regex4.java
# java Regex4
Match from 1 to 3
Match from 5 to 7
Match from 9 to 11
```

Figure 4: Output of `Regex3.java` and `Regex4.java`

`Matcher`. Consider the program `Regex5.java` shown below:

```
import java.util.regex.*;
```